

Verteilte System

Übungsblatt 2

Raphael Thelen 16.07.2025

Aufgabe 1

Implementierung

Aufgabe 1

Wahl der Programmiersprache

Java

- + Weit verbreitet
- + Plattformunabhängig
- + sim4da
- Veraltet
- Code lang und schlecht lesbar
- Uncool*

Kotlin

- + Weniger Verbreitet
- + Plattformunabhängig
- + 100% Java-kompatibel
- + Modern
- + Code kurz und prägnant
- + Fast so cool wie JS*
- + In IntelliJ integriert

* persönliche Meinung des Autors

Aufgabe 1

Kotlin im Detail

- Typinferenz

```
val number = 42
```

- Smart Casts

```
if (obj is String) {    println(obj.length)    }
```

- Extension Functions

```
val email = "foo@bar.com"  
email.isEmail()
```

Aufgabe 1

Null-Sicherheit in Kotlin

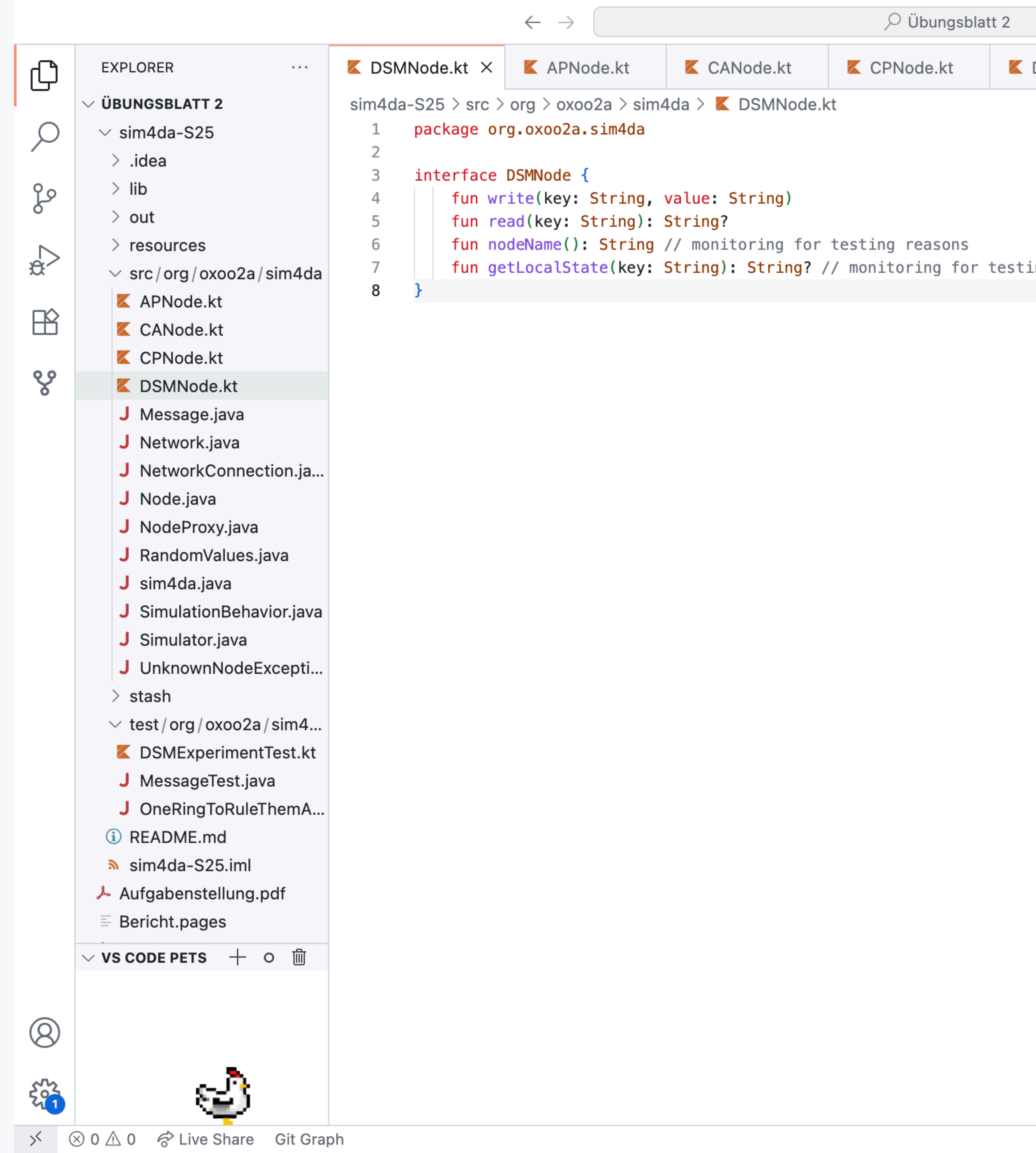
- Elvis-Operator `?:` Null Coalescing
`val first_name = name ?: "Elvis"`
- Weitere Operatoren
 - Not Null Assertion `!!`
 - Safe Call `?.`
 - Safe Cast `as?`

Laut Groovy Community
gibt es hier eine Ähnlichkeit

• ~
• •



Der Code



Aufgabe 2

Testprogramm

Aufgabe 2

Testprogramm

- Counter
- Vier Tests → Aufgabe 3
 - Keine Latenz Zu wenig bei Nicht-Auslastung der Verbindung
 - ~~Hohe Latenz~~ → Uniforme Nachrichtenauswahl statt FIFO
 - Partitionierung → Queue anpassen um Nachrichten zu Filtern
 - ~~Partitionierung mit hoher Latenz~~
- Externer Beobachter erstellt Snapshots

Aufgabe 3

Simulation

Aufgabe 3

Resultate

- Ohne Partition
 - AP: ca. 90% Konsistenz
 - CP: ca. 85%
 - CA: ca. 85%
- Mit Partition
 - AP: >90% Konsistenz
 - CP: ca. 90%
 - CA: ca. 90%

Analyse der Resultate

AP

- Ergebnisse entsprechen grob den Erwartungen
- Konsistenz ist mit Partition höher
 - ➔ Geänderte Berechnungsgrundlage



Analyse der Resultate

CP

- Ergebnisse entsprechen nicht den Erwartungen
- System sollte vollständig konsistent sein
 - ➡ Suboptimale Implementierung des Tests
 - Beobachter liest lokalen Zustand
Statt echtem Read-Zugriff
 - ➡ System sollte bei Partition blockieren
 - Timeout begünstigt Availability
Auf Kosten von Konsistenz



Analyse der Resultate

CP

- Ergebnisse entsprechen nicht den Erwartungen
- System sollte vollständig konsistent sein
 - ➡ Suboptimale Implementierung des Tests
 - Beobachter liest lokalen Zustand
 - Statt echtem Read-Zugriff
- ➡ CP und AC liefern identische Ergebnisse
- ➡ CP und AC sind in praktisch identisch



Fazit

Fazit

- Implementierung des Test suboptimal
 - Für Aussagekräftige Ergebnisse sollten echte read()-Aufrufe verwendet werden
- Implementierung von CP und AC suboptimal
 - write() sollte kein Timeout verwenden um Konsistenz nicht zu kompromittieren
 - CP läuft bei simultanem Lesen und Schreiben in einen Deadlock

Link zum Code



<https://github.com/Raphael-Thelen/Verteilte-Systeme>